

1. 概述

1.1. 编译过程的六阶段划分

把源程序编译为目标程序的过程可划分为六个阶段

- 词法分析：把一串普通的字符识别为编译语言里的单词符号
- 语法分析：将词法分析得到的单词序列组合成各类语法短语
- 语义分析：审查源程序的语义是否正确，如类型检查
- 中间代码生成：将语义分析后的结果翻译成一种标准且通用的形式
- 代码优化：在不改变程序语义的前提下，提高执行效率或减少资源消耗
- 目标代码生成：将（优化后的）中间代码转换为特定机器上的**目标代码**（汇编或机器指令）

1.2. 编译程序的结构

编译程序还必须包括辅助部分——符号表与出错处理

符号表参与编译的全过程，记录词法分析阶段所有的符号信息（如常量、标识符、函数名等）

出错处理负责检测、定位并报告源程序中出词法、语法、语义错误

编译原理也可以划分为前端和后端

- 前端：依赖于源程序而不是目标机。包括中间代码生成及之前的部分
- 后端：依赖于目标机而不是源程序

源程序经过前端就变成中间代码，经过后端变成源程序

2. 文法与语言

老师录音

文法推导过程，构建语法树

短语、简单短语、句柄。出现多个相同符号时记得写下标

文法是描述语法的形式化工具。它描述一个语言应该具有什么样的结构

2.1. 字母表与符号串

字母表是符号的非空有穷集合，可用于表示文法内允许出现哪些符号；符号串则是字母表中的符号组成的有穷序列。允许为空，用 ε 表示。其长度用绝对值符号表示

在 $z=xy$ 中， x, y, z 都是符号串， z 拆分为 x, y 。那么 x 是 z 的头， y 是 z 的尾。由于拆分方法不同，头、尾并不唯一，且可以为空 ε 。去除完整字符串 z 后，头、尾变成固有头、固有尾

符号串之间的连接是隐藏的。如 $x=abc, y=123$ ，则 $xy=abc123$

符号串用次幂表示重复， $x^0 = \varepsilon, x^2 = xx, x^n = xx^{n-1}$

符号串集合之间的乘积可以理解为集合论中的笛卡尔积

$$A = \{a, b\}, B = \{c, d\}$$

$$AB = ac, ad, bc, bd$$

再引入闭包的概念，一个符号串集合 A 通过任意次乘积，所能表达的符号串集合，记为正闭包 A^+ ，再加上空集则变成星闭包 A^*

2.2. 文法的形式定义

文法定义为四元组 $G = (V_N, V_T, P, S)$ ，其中：

- V_N 为非终结符的非空有穷集合，是一些抽象的、可以继续展开的语法成分，比如 表达式
- V_T 为终结符的非空有穷集合，表示语言中不可再分的基本符号。且 V_N, V_T 两者交集为空，并集 $V = V_N \cup V_T$ 也称字母表
- P 为产生式的非空有穷集合。基本形式为 $\alpha \rightarrow \beta$ ，左部 $\alpha \in V^*V_NV^*$ 表示至少包含一个非终结符，右部 $\beta \in V^*$
- S 为文法的开始符，出现在至少一条产生式的左部

书写符号的约定如下：

- 用大写字母表示非终结符， S, Z 一般用作开始符；若用文字版表示，则须用尖括号括住文字
- 用小写字母表示非终结符
- 第一条产生式的左部是开始符，或用 $G[S]$ 指出开始符为 S
- E (Expression) 表示表达式，T (Term) 表示项，F (Factor) 表示因子

文法的推导

如果一个字符串 $v = x\alpha y$ 可以通过产生式 $\alpha \rightarrow \beta$ 变成 $w = x\beta y$ ，则记为 $v \Rightarrow w$ ，称为直接推导。

若 v 能通过任意次（大于 0）推导到 w ，则记为 $v \Rightarrow^+ w$ ，称为推导，其长度表示推导的次数。若推导次数可以为 0，则成为广义推导，记为 $v \Rightarrow^* w$

如果推导的每一步都是替换字符串最左的非终结符，则称每一步推导都是最左直接推导 $v \Rightarrow_L w$ ，整个推导记为最左推导。同理，还可以给出最右推导。最右推导也称**规范推导**，最左推导的逆过程称为规范规约

由文法定义的语言

有文法 $G[E]$ ，若 $E \Rightarrow^* x$ ，即符号串 x 可以从开始符推导出来，则称 x 为 $G[E]$ 的句型。仅包含终结符的句型称为句子，即 $E \Rightarrow^* x \wedge x \in V^*$ ，由最左（右）推导出的句型称为左（右）句型，右句型也称为规范句型

文法 $G[E]$ 定义的**语言**是从 E 推导出的全部句子的集合，用 $L(G)$ 表示。如果两个文法的语言 $L(G1) = L(G2)$ 是相同的，则称文法 $G1, G2$ 等价

文法唯一确定一种语言，语言可能对应多种文法

递归规则与递归文法

如果一个规则的左右出现了相同的非终结符，则称为递归规则。

$A \rightarrow Ay$ 称为左递归规则； $A \rightarrow xA$ 称为右递归规则； $A \rightarrow xAy$ 称为内递归规则

如果文法中至少包含一条递归规则，则称文法是**直接递归**的；如果文法中可以通过推导得到递归的结果 $A \Rightarrow^+ xA|Ay|xAy$ ，则称文法为间接左递归或间接右递归或间接内递归

如果文法中存在 $A \Rightarrow^+ xAy$ ，则该文法是递归文法

文法的类型

0 型文法：最一般的文法，只要求产生式左部至少含有一个非终结符

1 型文法（上下文有关文法）：其产生式满足 $xAy \rightarrow x\beta y$ ，其中 $A \in V^*$, $\beta \in V^+$ ， xy 就是上下文

2 型文法（上下文无关文法）：其产生式满足 $\alpha \rightarrow \beta$, $\alpha \in V^N$, $\beta \in V^*$ ，即产生式的左部均为单独的非终结符。常用的程序设计语言都是 2 型文法

3 型文法（正规文法）：其产生式满足如下两种条件之一

- 只包含 $A \rightarrow aB$ 或 $A \rightarrow a$, $A, B \in V^N$, $a \in V^T$ ，右线性文法
- 只包含 $A \rightarrow Ba$ 或 $A \rightarrow a$, $A, B \in V^N$, $a \in V^T$ ，左线性文法

这四个文法之间是逐级包含的关系

2.3. 语法树

给定文法 $G[S]$ ，对于任何句型都能构建与之关联的语法树（推导树），满足如下条件

- 这棵树的根为 S
- 非叶子节点只能是非终结符
- 叶子节点是非终结符或终结符
- 从节点到孩子节点，必定存在产生式

例题

$$\begin{aligned} G[E] : E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

构造句型 $id * id + id$ 的语法树

（手写）

短语、句柄

一个句型唯一确定语法树，其所有叶子组成的符号串就是一个**短语**。也可以理解为，短语就是语法树（推导过程）中，某个非终结符对应到句型的一段串

如果某个短语是由一个非终结符一步推出的，那么它就是**简单短语**，最左边的简单短语就是**句柄**

以刚才的例题为例，短语包括 $id_1 * id_2 + id_3$ 、 $id_1 * id_2$ 、 id_1 、 id_2 、 id_3 ，简单短语包括 id_1 、 id_2 、 id_3 ，句柄是 id_1

二义性

给定一个文法，如果存在某个句子对应两颗不同的语法树或，者存在某个句子对应两种不同的最左/最右推导，则该文法是二义性的

2.4. 句型分析

句型分析是一个识别输入符号串是否符合语法的过程

可以有自上而下和自下而上两种分析法

2.5. 文法的实用限制

$U \rightarrow U$ 称为有害规则；对于 $U \rightarrow u$ ，如果 U 是不可到达（无法推导得到）或使用该规则后再也推导不出终结符，则称为多余规则

文法的实用限制包括不能存在有害规则和多余规则。称符合实用限制的文法是压缩过的

实际处理中还可以有以下限制：

- 开始符不在规则右部
- 不存在 $A \rightarrow B$
- 不存在左递归规则 $A \rightarrow Aa$
- 不存在空规则 $A \rightarrow \varepsilon$

任意一个 2 型文法都能被构造符合上述要求的等价文法，步骤如下：

1. 引入新的开始符（满足 开始符不在规则右部 ）
2. 消除无用规则

看不懂，跳过

3. 词法分析

老师录音

确定化和最小化。特别关注最小化

注意最小化中找出多余状态的过程、初态、终态

根据自动机写正规式

3.1. 词法分析程序

分类：标识符（函数/变量的名称）、关键字（语言保留字）、常量、运算符、界符（如 `(){}|`）

词法分析程序输出的单词符号通常用二元组（单词类别，单词自身的值）表示

3.2. 正规式

正规式用于形式化描述正规语言（与正则表达式类似）

* 表示闭包，即任意有限次的重复； . 表示连接，一般可省略； | 表示或，即二选一

正规式	正规集
$(a b)(a b)$	$\{aa, ab, ba, bb\}$
$(a b)^*$	$\{\varepsilon, aa, ab, ba, bb, \dots\}$

题目中，会考正规式与正规集的互推
如果两个正规式对应的正规集相同，则两个正规式等价

正规文法构造正规式

对于正规文法规则 $U \rightarrow a_1W_1|\dots|a_2W_2|b_1|\dots|b_m$ ，其等价正规式方程为 $U = (b_1 + \dots + b_m) + a_1W_1$ 。这里的加法 $+$ 表示并集
若 $X = aX + b$ ，则可以写成 $X = a^*b$

题目中，会要求用正则式描述文法 G 定义的语言 $L(G)$
可以把多条生成式逐条转化为正则式方程组，再解方程组

3.3. 确定的有穷自动机 DFA

一个确定的有穷自动机（DFA）是一个五元组 $M = (L, V_T, f, S, Z)$ ，与数字电路中的状态机概念类似。
其中， K 表示有穷的状态（图论中的点）； V_T 是有穷的输入字母表； $f, K \times V_T \rightarrow K$ 是状态转移函数，也可记为 $f(W, a) = U$ ； $S \in K$ 是唯一的初态； Z 是终止态集，是 K 的非空子集
这里的 **确定** 指的是一个状态下面对一个输入，有唯一确定的下一个状态
DFA 可以用状态机和矩阵表示。在状态机中，初态旁边标负号 $-$ ，终态旁边标正号 $+$ 。在矩阵中，行表示当前状态，列表示输入字母

有穷自动机的识别

如果 DFA 上有一条从初态达终态的路径，并将这条路径上所有弧的标记连接起成符号串 x ，则称 x 能被 DFA 所接受，即这个符号串是文法定义的语言。如果初态、终态是同一种状态，则表示空 ε 能被识别
DFA 能识别的字符串集合称为 DFA 所接受的语言，记为 $L(DFA)$

3.4. 不确定的有穷自动机

一个不确定的有穷自动机（NFA）是一个五元组 $M = (L, V_T, f, S, Z)$
其中，只有 f 的定义和 DFA 不同。
这里的 **不确定** 指的是一个状态下面对一个输入，有多个可选的下一个状态。即 $f(W, a) = \{k|k \in K \wedge \text{其他条件}\}$
类似的，NFA 可以用状态机和矩阵表示。其所接受的语言也和 DFA 使用相同的定义

3.5. 确定化：NFA->DFA

使用子集构造法，用 DFA 的一个状态来表示 NFA 的一个状态集合，可以把 NFA 转化为 DFA

ϵ -closure

设 $I \subseteq K$, ϵ -闭包 $\epsilon - closure(I)$ 表示从任意状态 $i \in I$ 出发, 只通过 ϵ 边, 也就是不消耗任何输入字符, 能够到达的所有状态集合

move(I,a)

a 弧转换 $move(I, a)$ 表示从任意状态 $i \in I$ 词法, 读入字符 a 后, 能够到达的状态集合

$I_a = \epsilon - closure(move(I, a))$, 即弧转换后的 ϵ -闭包

子集构造法的步骤

1. 计算 NFA 初始状态的 ϵ -闭包, 将其视为 DFA 的初始状态集合
2. 对每个已有的 DFA 的状态, 遍历每个输入符号, 计算其转移后的 ϵ -闭包 I_a 。也将其视为 DFA 的状态
3. 不断重复第 2 步, 直到没有新状态产生
4. 将包含 NFA 终态的 DFA 状态作为 DFA 终态

在作答时, 可以使用表格形式

I	$move(I, a)$	I_a	$move(I, b)$	I_b
I0	...	I1	...	I2
I1				
I2 (用队列管理待处理的 DFA 状态)				

3.6. 最小化: DFA->miniDFA

miniDFA, 需要满足两个条件: 没有多余状态 (无法到达或无法终止的状态); 没有等价状态

状态 s, t 等价的必要条件: s, t 都是终态或都是非终态; 对于所有输入符号, s, t 转移到相同状态

分割法计算 miniDFA

1. 将所有状态划分为互斥的两个集合: 终态集和非终态集。所有终态都划分到终态集
2. 划分集合后, 检查同一个集合中的状态在相同输入符号下的转移情况。如果不同状态转移到不同状态, 则这些状态不等价, 需要分割
3. 直到所有集合都无法再被分割, 这些集合就称为等价类
4. 从每个等价类中选一个状态作为代表, 重新构建出的就是 miniDFA

3.7. 正规式与有穷自动机的等价性

(RE->NFA) 对于任意正规式 e , 存在一个与之相应的 NFA M , 使得 $L(M) = L(e)$; (NFA->RE)

对于任意 NFA M , 存在一个与之相应的正规式 e , 使得 $L(e) = L(M)$

DFA 转 RE 直接手撕即可

RE→NFA

(见书 P53, 下面空位手写补充)

空集 $\emptyset$

空字符串

终结符 a

非终结符 PQ

非终结符 $P|Q$

P^*

3.8. 正规文法与有穷自动机的等价性

3G→NFA

对于右线性文法 $U \rightarrow a|aW$

1. 每个非终结符画一个阶段, 开始符 S 为开始状态
2. 增加一个终止状态
3. 对 $U \rightarrow a$, 画一条从 U 到 Z 的 a 弧; 对 $U \rightarrow aW$, 画一条从 U 到 W 的 a 弧

对于左线性文法 $U \rightarrow a|Wa$, 上述过程需要发生一些改变: 开始符 S 为终止状态, 其余不难, 自己补全

NFA→3G

1. $f(A, t) = B$ 转化为产生式 $A \rightarrow tB$
2. 终止状态 Z 增加产生式 $Z \rightarrow \varepsilon$
3. NFA 初态对应开始符, 状态集对应非终结符号集, 输入字母表对应终结符